

作业 10

——高维欧式空间最近邻查询——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。

分享这个文档仅限于学术交流，不用于任何形式的商业用途。

1. 题目

描述

给定 d 维空间中的 n 个点组成的集合 P ，点从 0 开始依次编号。你的程序需要回答 q 次询问，每次询问给出一个 d 维空间中的点，询问该点在集合 P 中的近似最近邻，距离定义为欧氏距离。¹ 保证所有给出的点都位于单位球面上。

具体地，对于每次询问，你需要输出一个 0 到 $n-1$ 中的整数表示你找到的近似最近邻点在集合 P 中的编号，要求输出的点到询问点的距离不能超过最优解的 4 倍。

为了方便输入，在输入数据中给出的是坐标在 $[-100, 100]$ 的方向向量，对应的真正的数据点需要正则化成单位球面上的点。 $1 \leq d \leq 64, 1 \leq n \leq 10000, 1 \leq q \leq 20000$ ，保证 d 是 2 的次幂。保证 P 中任意两点要么坐标相同，要么间距大于 0.6。

输入

第一行三个正整数 d, n, q 。之后 n 行，每行 d 个整数，依次表示 P 中一个点的坐标。之后 q 行，每行 d 个整数，表示一个询问。

输出

对于每组询问，你需要输出一行一个整数，表示近似最近邻的下标。

样例输入

```
4 5 5
-97 -95 86 49
26 39 -47 -68
8 4 73 96
-89 -94 37 91
-99 -12 84 -71
26 39 -47 -68
26 39 -47 -68
-89 -89 39 82
19 31 -40 -61
8 5 66 87
```

样例输出

```
1
1
3
1
2
```

¹设点 $A(a_1, a_2, \dots, a_d), B(b_1, b_2, \dots, b_d)$, $\text{dist}(A, B) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_d - b_d)^2}$

2. 思路分析

这道题目的思路是，我们不能计算两个点之间的距离，所以还是需要使用哈希函数，把相近的点能够映射到同一个桶里面，然后只需要找到这个桶里面的点就可以了。

我们现在关心的就是如何设计这样的一个哈希函数，能够将一个 d 维的向量进行映射。这里涉及比较复杂的数学知识（至少对于我来说比较复杂）。大概方法是经过 Hadamard 变换之后，随机旋转，然后根据各个旋转之后的向量在坐标轴方向的分量大小来进行哈希映射。接下来是主要的步骤：

2.1. 归一化

将输入的坐标进行归一化处理，使其位于单位球面上，这样，两个点之间的距离只与它们的夹角有关系。假设两个点之间的夹角是 θ ，由余弦定理，

$$d^2 = a^2 + b^2 - 2ab \cos \theta = 2 - 2 \cos \theta$$

$$\langle \alpha, \beta \rangle = \frac{|\alpha| \cdot |\beta|}{\alpha \cdot \beta} = \frac{1}{\prod_{i=1}^d (a_i \cdot b_i)} = \cos \theta. (\alpha = (a_1, a_2, \dots, a_d), \beta = (b_1, b_2, \dots, b_d))$$

为了实现归一化，我们设计这个函数，其中， d 代表维度， n 代表需要处理的点的数量。这里是可以只运行一次函数就处理完成所有的点。

```
void normalize(int d, int n, vector<vector<double>> &points) {
    for (int i = 0; i < n; i++) {
        double len = 0;
        for (int j = 0; j < d; j++)
            len += points[i][j] * points[i][j]; // 计算平方和
        len = sqrt(len); // 计算向量长度
        for (int j = 0; j < d; j++)
            points[i][j] /= len; // 除以长度，变成单位向量
    }
}
```

2.2. 随机旋转

大概意思是我们可以从不同的角度去观察归一化之后单位球上的各个坐标，但是交给程序我们需要进行线性变换，相当于把坐标轴进行了旋转，我们只需要保证这个矩阵是单位正交矩阵，就能不改变向量之间的关系。

```
vector<double> fastHadamardTransform(vector<double> x) {
    int m = x.size();
    for (int step = 1; step < m; step <= 1) {
        for (int i = 0; i < m; i += 2 * step) {
            for (int j = 0; j < step; j++) {
                double u = x[i + j];
                double v = x[i + step + j];
                x[i + j] = u + v;
                x[i + step + j] = u - v;
            }
        }
    }
    double norm = 1.0 / sqrt((double)m);
    for (int i = 0; i < m; i++)
        x[i] *= norm;
    return x;
}

vector<int> randDiag(int d) {
    vector<int> diag(d);
    for (int i = 0; i < d; i++) {
```

```

        // rand()%2生成0或1, 乘以2减1得到-1或+1
        diag[i] = (rand() % 2) * 2 - 1;
    }
    return diag;
}

struct FastRotation {
    vector<int> D1, D2, D3; // 三个随机对角矩阵
    int dim;
    FastRotation(int d) {
        dim = d;
        D1 = randDiag(d);
        D2 = randDiag(d);
        D3 = randDiag(d);
    }
    vector<double> rotate(vector<double> x) {
        x = fastHadamardTransform(x);
        for (int i = 0; i < dim; i++) x[i] *= D1[i];
        x = fastHadamardTransform(x);
        for (int i = 0; i < dim; i++) x[i] *= D2[i];
        x = fastHadamardTransform(x);
        for (int i = 0; i < dim; i++) x[i] *= D3[i];
        return x;
    }
};

```

2.3. 计算哈希值

这里的哈希函数经历了如下过程：首先，对原来的单位球进行随即旋转，然后找到绝对值最大的坐标，并根据符号编码到奇数和偶数。接下来由 compound 哈希来进行整合。由于之前的哈希函数每次只能得到一个坐标的哈希值，所以我们需要多个哈希函数来得到一个更长的哈希值，这样才能更好地区分不同的点。你可以理解成对这个向量，我们考察了它的多种特征，从不同视角观察得到不同的特征，我们断言，通过这些特征能够足以找出相近的向量了。

```

struct CrossPolyHash {
    FastRotation rot;
    int dim;
    CrossPolyHash(int d) : rot(d) {
        dim = d;
    }
    int operator()(const vector<double> &point) {
        vector<double> y = rot.rotate(point);
        // 找到绝对值最大的坐标
        int best = 0;
        double bestAbs = fabs(y[0]);
        for (int i = 1; i < dim; i++) {
            double cur = fabs(y[i]);
            if (cur > bestAbs) {
                bestAbs = cur;
                best = i;
            }
        }
        // 根据符号返回哈希值：正返回2i, 负返回2i+1
        if (y[best] > 0)
            return 2 * best;
        else
            return 2 * best + 1;
    }
};

```

```

// 复合哈希：将K个单个哈希函数组合成一个长哈希
struct CompoundHash {
    vector<CrossPolyHash> hashes;
    int K;

    CompoundHash(int d, int K_val) {
        K = K_val;
        for (int i = 0; i < K; i++) {
            hashes.push_back(CrossPolyHash(d));
        }
    }

    unsigned long long encode(const vector<double> &point) {
        unsigned long long key = 0;
        for (int i = 0; i < K; i++) {
            int val = hashes[i](point);
            // 每个哈希值占7位，左移7位后拼接
            key = (key << 7) | (val & 0x7F);
        }
        return key;
    }
};

```

至此，我们所需要的一切工具就准备好了。

接下来只需要处理输入输出就好咯~

3. 完整程序

```

int main() {
    srand((unsigned int)time(0)); // 设置随机种子
    int d, n, q;
    cin >> d >> n >> q;
    // 读取数据点
    vector<vector<double> > points(n, vector<double>(d));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < d; j++) {
            cin >> points[i][j];
        }
    }
    normalize(d, n, points);
    // 读取查询点
    vector<vector<double> > queries(q, vector<double>(d));
    for (int i = 0; i < q; i++) {
        for (int j = 0; j < d; j++) {
            cin >> queries[i][j];
        }
    }
    normalize(d, q, queries);
    // LSH参数：L个哈希表，每个表用K个哈希函数
    const int L = 24;
    const int K = 8;
    // 初始化L个复合哈希函数和L个哈希表
    vector<CompoundHash> compoundHashes;
    vector<map<unsigned long long, vector<int> > > tables(L);
    for (int i = 0; i < L; i++) {
        compoundHashes.push_back(CompoundHash(d, K));
    }
}

```

```
// 构建索引：将所有数据点插入到L个哈希表中
for (int idx = 0; idx < n; idx++) {
    const vector<double> &pt = points[idx];
    for (int i = 0; i < L; i++) {
        unsigned long long key = compoundHashes[i].encode(pt);
        tables[i][key].push_back(idx);
    }
}

// 用于标记已经检查过的点，避免重复计算
vector<int> seen(n, 0);
int queryID = 0;
// 处理每个查询
for (int t = 0; t < q; t++) {
    queryID++;
    const vector<double> &qpt = queries[t];
    vector<int> candidates;
    // 从L个哈希表中收集候选点
    for (int i = 0; i < L; i++) {
        unsigned long long key = compoundHashes[i].encode(qpt);
        map<unsigned long long, vector<int> >::iterator it = tables[i].find(key);
        if (it != tables[i].end()) {
            const vector<int> &bucket = it->second;
            for (int j = 0; j < bucket.size(); j++) {
                int idx = bucket[j];
                if (seen[idx] != queryID) {
                    seen[idx] = queryID;
                    candidates.push_back(idx);
                }
            }
        }
    }

    // 在候选点中找最近邻
    int bestIdx = -1;
    double bestDist2 = 1e100;
    for (int i = 0; i < candidates.size(); i++) {
        int idx = candidates[i];
        double dist2 = 0.0;
        for (int j = 0; j < d; j++) {
            double diff = qpt[j] - points[idx][j];
            dist2 += diff * diff;
        }
        if (dist2 < bestDist2) {
            bestDist2 = dist2;
            bestIdx = idx;
        }
    }
    cout << bestIdx << endl;
}
return 0;
}
```

感谢昨天请我喝咖啡的同学，看看今天有人请我不